SOFTWARE PROJECT COMPLETION REPORT

Prepared by: Amanjyot Singh Bagga
Date: June 22, 2026
Project Name: In/Out Attendance & Wage Automation System
Project Status: Completed

TABLE OF CONTENTS

---

1 EXECUTIVE SUMMARY

This report documents the successful completion of the In/Out Attendance & Wage Automation System, a web-based application built on Google Apps Script and Google Sheets. The system enables employees to record IN and OUT punches with GPS coordinates, device information, geofence validation, and automated wage calculation including overtime.

The solution operates entirely within the Google Workspace ecosystem and does not require external hosting infrastructure. It provides attendance logging, monthly sheet management, device anomaly detection, and automated monthly reporting for HR and finance teams.

2 PROJECT OVERVIEW

Table 1: Project Overview

Field	**Details**
Project Name	In/Out Attendance & Wage Automation System
Platform	Google Apps Script Web App
Frontend	HTML5, CSS3, Vanilla JavaScript
Backend	Google Apps Script
Data Store	Google Spreadsheet
Core Features	IN/OUT attendance, geofence validation, device anomaly detection, wage calculation, monthly summary
Target Users	Internal Operations, HR, Finance
Deployment	Google Apps Script Web App
Location Context	Indore, Madhya Pradesh, India
Time Zone	IST (Asia/Kolkata)

3 BUSINESS REQUIREMENTS

3.1 Problem Statement

The organization previously relied on manual attendance recording and separate wage calculations. This approach was time-consuming, error-prone, and vulnerable to misuse such as fake attendance and off-site punches.

3.2 Functional Requirements

Table 2: Functional Requirements

Req. ID	Requirement	Priority
BR-01	Allow employees to record IN attendance via web app	High
BR-02	Allow employees to record OUT attendance via web app	High
BR-03	Capture GPS coordinates for each punch	High
BR-04	Enforce geofence around office location	High
BR-05	Capture device information including browser, OS, type, and model (with user consent)	Medium
BR-06	Detect anomalous device usage per employee	High
BR-07	Maintain per-month attendance sheets with detailed logs	High
BR-08	Automatically compute total, normal, and overtime hours per OUT	High
BR-09	Calculate daily wages using configurable hourly and overtime rate	High
BR-10	Generate monthly wage summary per employee and grand total	High
BR-11	Email monthly attendance sheet to admin	Medium
BR-12	Allow HR to edit IN/OUT times and recalculate wages automatically	High
BR-13	Maintain list of active employees with hourly rate configuration	High
BR-14	Cache active employees and monthly rates to improve performance	Medium
BR-15	Highlight geofence and device anomalies directly in the sheet	Medium
BR-16	Provide hyperlink to location in Google Maps for each punch	Low

3.3 Non-Functional Requirements

Table 3: Non-Functional Requirements

Req. ID	Requirement
NFR-01	System must work within Google Apps Script quotas and limits
NFR-02	UI must be mobile-responsive and easy to use
NFR-03	Backend must handle typical daily attendance volumes reliably
NFR-04	All data stored in a single controlled spreadsheet for auditability

4 TECHNICAL REQUIREMENTS

4.1 Google Services Required

Table 4: Google Services Used

Service	Purpose
SpreadsheetApp	Read/write monthly attendance sheets and Employees config
ScriptApp	Manage triggers, OAuth, and execution context
PropertiesService	Store active employee list, monthly rate snapshots, and device state
UrlFetchApp	Export monthly sheet as XLSX and fetch via export URL
MailApp	Send monthly attendance reports to admin
HtmlService	Serve the web app frontend
Utilities	Date formatting and time zone handling

4.2 Frontend Technologies

- **HTML5** for form structure, dropdowns, and buttons.
- **CSS3** for responsive layout and styling.
- **Vanilla JavaScript** for client-side logic and server calls.
- **UAParser.js** for browser, OS, and device parsing.
- **User-Agent Client Hints API** for modern device model detection.
- **Geolocation API** for latitude and longitude capture.
- **Intl.DateTimeFormat** for live IST clock display.

4.3 Backend Functions Implemented

- `doGet()` for serving the web app and ensuring setup.
- `ensureOnEditTrigger()` for installing the edit trigger.
- `getOrCreateEmployeesSheet()`, `syncActiveEmployeesCache()`, `getActiveEmployees()`.
- `snapshotEmployeeRates()`, `getRateFromCache()`.
- `getMonthlySheet()`, `emailMonthlySheet()`.
- `checkDeviceAnomaly()`, `processSubmission()`.
- `calculateWages()`, `generateMonthlySummary()`, `onSheetEdit()`.
- Helper functions: `formatHoursMinutes()`, `parseTime()`, `isInsideGeofence()`, `toRad()`.

5 SYSTEM ARCHITECTURE

The solution follows a browser–Apps Script–Spreadsheet architecture:

- **User Browser**: Hosts the HTML/CSS/JavaScript frontend with form UI, geolocation capture, and device info collection.
- **Google Apps Script Backend**: Implements server-side logic including trigger management, geofence checks, anomaly detection, wage calculation, and reporting.
- **SpreadsheetApp Data Store**: Stores the Employees sheet, monthly attendance sheets, and summary rows.

6 DEVELOPMENT PROCESS

6.1 Methodology

The project followed an iterative development approach with continuous feedback loops.

Phase 1 - Requirements Gathering

- Identified attendance and wage calculation pain points.
- Defined core attendance fields and report structure.
- Finalized geofence center, radius, and wage policy.

Phase 2 - UI/UX Design

- Designed a mobile-friendly interface.
- Implemented a name dropdown and IN/OUT buttons.
- Added live clock display in IST.

Phase 3 - Backend Development

- Built `doGet()` as the entry point.
- Implemented monthly sheet creation and summary generation.
- Added geofence and device anomaly detection.

Phase 4 - Frontend Integration

- Implemented geolocation capture and device detection.
- Connected frontend to backend via `google.script.run`.
- Added success and failure handlers.

Phase 5 - Smart Features

- Added monthly rate snapshots and optional email reporting.
- Enabled automatic wage recalculation when times are edited.

Phase 6 - Testing and Bug Fixes

- Verified geofence, device anomaly, and wage logic.
- Fixed date formatting and edge case issues.

Phase 7 - Deployment

- Deployed as a Google Apps Script Web App.
- Completed pilot rollout and validation.

7 FEATURES IMPLEMENTED

7.1 Core Features

Table 5: Core Features

Feature	Status
IN and OUT attendance recording via web app	Done
Name dropdown for active employees	Done
GPS capture	Done
Geofence evaluation	Done
Monthly sheet creation	Done
Employee hourly rate configuration	Done
Daily hours and wage calculation	Done
Monthly wage summary per employee	Done
Grand total wage row per month	Done

7.2 Smart / Automated Features

Table 6: Smart Features

Feature	Status
Auto-install onEdit trigger	Done
Active employee cache	Done
Monthly rate snapshot	Done
Automatic monthly summary generation	Done
XLSX export and email	Done
Automatic wage recalculation on time edits	Done
Geofence coloring in sheet	Done
Device anomaly highlighting	Done
Google Maps hyperlink per punch	Done

8 TESTING & QUALITY ASSURANCE

8.1 Test Cases Executed

A comprehensive set of test cases was executed covering geofence boundary conditions, device anomaly scenarios, wage calculations, monthly summaries, onEdit behavior, frontend submissions, and error handling.

8.2 Bug Log and Fixes

Representative issues addressed included:

- Date formatting inconsistencies in Sheets.
- Wage calculation failures when rate cache was missing.
- Negative duration handling when IN/OUT were misordered.

9 PERFORMANCE TESTING

Table 7: Performance Metrics

Metric	Target	Observed	Status
Web app initial load time	< 3 seconds	~ 2.0 seconds	Pass
Submit IN/OUT time	< 5 seconds	$\sim 1.5 - 2.5$ seconds	Pass
Wage calculation per OUT	< 2 seconds	$\sim 0.5 - 1.0$ seconds	Pass
Monthly summary generation	< 15 seconds	$\sim 5 - 10$ seconds	Pass
Active employee cache refresh	< 3 seconds	~ 1.0 second	Pass

10 BROWSER COMPATIBILITY TESTING

Table 8: Browser Compatibility

Browser	Version Tested	Desktop	Mobile
Google Chrome	122+	Fully Compatible	Fully Compatible
Microsoft Edge	121+	Fully Compatible	Fully Compatible
Mozilla Firefox	123+	Fully Compatible	Fully Compatible
Safari	17+	Fully Compatible	Fully Compatible
Chrome (Android)	122+	–	Fully Compatible
Safari (iOS)	17+	–	Fully Compatible

11 SECURITY TESTING

Security-related checks included:

- Validation that only configured employees appear in the dropdown.
- Geofence enforcement with inside/outside highlighting.
- Device anomaly detection with threshold-based acceptance.
- Defensive handling of malformed device information and missing geolocation.
- Isolation of each user submission.

12 DEPLOYMENT

12.1 Deployment Steps

1. Create a new Apps Script project.
2. Paste backend code into `Code.gs`.
3. Create `Index.html` and paste the frontend code.
4. Set spreadsheet ID and configuration constants.
5. Deploy as Web App.
6. Authorize required scopes.
7. Share the web app URL/ QR Code with employees.

12.2 Deployment Configuration

Table 9: Deployment Configuration

Setting	**Value**
Project Name	In/Out Attendance & Wage Automation System
Execute As	Me (Developer Account)
Who Has Access	Anyone in organization / allowed users
Deployment Type	Web App
Environment	Google Apps Script Cloud
External Hosting Required	None
Monthly Hosting Cost	Rs 0

13 CHALLENGES & SOLUTIONS

Table 10: Challenges and Solutions

#	**Challenge Faced**	**Solution Implemented**
1	Persistent device identification across sessions	Used localStorage device ID and backend device anomaly logic
2	Geofence enforcement in browser environment	Used Geolocation API and haversine formula on backend
3	Date formatting inconsistencies in Sheets	Forced date column to plain text and used explicit formatting
4	Wage calculation for edited times	Added robust time parsing and IN lookup validation
5	Performance and quotas under load	Used caching and minimized repeated spreadsheet reads
6	Monthly report generation without manual export	Implemented XLSX export and email automation

#	**Challenge Faced**	**Solution Implemented**
7	onEdit recalculation complexity	Built selective recalculation logic in trigger handler
8	Normal vs overtime split logic	Encoded configurable daily limit and overtime multiplier

14 KNOWN LIMITATIONS

Table 11: Known Limitations

#	Limitation	Severity	Recommendation
1	Apps Script max execution time is limited	Medium	Keep monthly sheets reasonable in size
2	Employee identity based on name only	Medium	Introduce unique employee ID
3	Device anomaly detection relies mainly on device ID	Medium	Extend with IP and more fingerprint fields
4	No built-in app-level role-based access control	Medium	Add RBAC in future version
5	Single geofence center configuration	Low	Support multi-office geofences later
6	No rollback after sheet writes	High	Add backup tabs before bulk changes

15 FUTURE RECOMMENDATIONS

15.1 High-Priority Recommendations

1. Introduce unique employee IDs as primary keys.
2. Add automatic backup and rollback mechanism.
3. Implement role-based access control.
4. Build an anomaly dashboard for HR/admin review.

15.2 Medium-Priority Recommendations

1. Support multi-office geofence locations.
2. Add extended audit logging.
3. Create notifications for repeated anomalies or missing OUT punches.
4. Add more validation rules such as break policy checks.
5. Improve UX with punch history and status feedback.
6. Expand documentation and in-app help.

15.3 Low-Priority Recommendations

1. Add branding and theme customization.
2. Create department-specific dashboard tabs.
3. Provide multi-language support.

4. Schedule reminder emails for month-end events.
5. Export configuration and logs as an audit package.

16 PROJECT METRICS SUMMARY

16.1 Development Metrics

- 7 iterative phases from requirements to deployment.
- Complete Google Workspace-based stack.
- 15+ backend functions covering trigger management, caching, geofence, anomalies, wage computation, and reporting.

16.2 Testing Metrics

- Functional, performance, compatibility, and security validations completed.
- Core workflows verified end-to-end.

16.3 Cost Analysis

- Infrastructure cost: Rs 0.
- Additional licensing cost: Rs 0.
- Ongoing maintenance cost: Minimal.

16.4 Business Impact Metrics

- Reduced manual effort in attendance consolidation and wage calculation.
- Improved reliability through automated validation and reporting.
- Stronger control via geofence and device anomaly detection.

17 SIGN-OFF AND APPROVALS

The system has been developed, tested, and deployed as per the agreed scope and is ready for business use.

17.1 Handover Checklist

1. Source code handover.
2. Configuration and environment details.
3. Functional specification and requirements document.
4. Technical design and architecture notes.
5. User guide and SOPs.

6. Test cases and execution summary.
7. Known limitations and roadmap.
8. Access and permission matrix.
9. Support and escalation process.
10. Backup and data retention approach.
11. Formal sign-off from stakeholders.

18 CONCLUSION

The In/Out Attendance & Wage Automation System has successfully achieved its objectives of automating attendance recording, enforcing geofence and device-based controls, and enabling accurate wage reporting for HR and finance.

With a stable v1.0 release in production and a clear roadmap for future enhancements, the application is positioned to become the standard attendance and wage tracking platform within the organization.